FleetSync Enterprise

Vehicle Service & Maintenance Tracker — Comprehensive Engineering Notes

Author / Role: Engineering & Internship Team	**Backend:** Python 3.10+ & Flask	**Database:** MySQL 8.0 (XAMPP)
Document Version: 1.0 (Comprehensive)	**Local Server:** http://127.0.0.1:5000	**Target Audience:** Developers / Interns

1. Executive Project Overview

FleetSync is an enterprise web application designed for automotive service centers, dealerships, and fleet maintenance managers. It replaces manual paperwork with a centralized digital dashboard tracking vehicles, corporate customers, scheduled maintenance intervals, repair work orders, and billing invoices.

Core Capabilities:

- **Fleet & Vehicle Inventory:** Records VIN, registration plate, odometer, fuel type, engine specs, and vehicle photos.
- **Maintenance Interval Engine:** Calculates dynamic due dates based on kilometer intervals and months elapsed; flags 'Due Soon' and 'Overdue' statuses.
- **Work Orders & Service Records:** Multi-item repair invoices detailing individual parts, labor hours, technician assignment, tax calculations, and status transitions.
- **Role-Based Access Control:** Strict separation between Administrators (financial reports, system setup) and Technicians (work order execution).
- **Automated Reminders:** Notifications queue for upcoming or overdue services.

2. System Architecture & Request Lifecycle

The application follows a clean **Model-View-Controller (MVC)** architectural pattern using Python Flask:

Layer	Component	Role & Responsibilities
Presentation (Client)	Web Browser (Chrome/Edge)	Renders responsive HTML5/CSS3, executes client-side charts (Chart.js) and triggers AJAX API requests.
Routing & Controller	Flask App (app.py)	Parses HTTP GET/POST requests, enforces authentication/session guards, invokes business logic, and renders Jinja2 templates.
Data Access Layer	DB Helper (db.py)	Manages MySQL connections, executes safe parameterized queries, returns dictionary records, and handles commits/rollbacks.
Storage Layer	MySQL Server (XAMPP)	Listens on Port 3306; maintains ACID relational integrity across 7 normalized database tables.

3. Demystifying 'http://127.0.0.1:5000'

A common point of confusion for new developers is understanding the URL used to access local applications. Every local development address consists of three distinct elements:

URL Element	Technical Meaning	Real-World Analogy

http://	**HyperText Transfer Protocol:** The communication rules governing how browsers request and receive web documents.	The agreed language (e.g. English) used between two people talking.
127.0.0.1 (or localhost)	**Loopback IP Address:** A globally reserved network address pointing exclusively to *this exact physical computer*. Data packets never leave the machine.	The street address of the building you are currently standing inside.
:5000	**Port Number:** A virtual channel (1-65535) allocated to a specific running software process. Flask listens on port 5000 by default.	The specific apartment or room door number inside that building.

Port Distinction in this Project:

• **Port 5000:** Handled by Python/Flask (delivers the web pages).

• **Port 3306:** Handled by MySQL in XAMPP (delivers the database queries).

4. Database Architecture & Schema Design

The system uses a relational database schema named fleetsync_db, running on MySQL. The schema guarantees data integrity via foreign keys, cascading constraints, and indexed identifiers.

Table Name	Primary Key & Foreign Keys	Purpose & Key Attributes
users	id (PK)	Staff credentials, name, email, hashed password (bcrypt), role ('Admin', 'Technician').
customers	id (PK)	Vehicle owners & corporate fleet accounts (name, email, phone, company, address).
vehicles	id (PK) customer_id → customers(id)	Vehicle records: VIN, license plate, make, model, year, fuel type, odometer, image path.
maintenance_schedules	id (PK) vehicle_id → vehicles(id)	Scheduled services: service type, interval km/months, last performed date, next due date, status.
service_records	id (PK) vehicle_id, technician_id	Work orders: invoice number, service type, labor cost, parts cost, tax, total, payment status.
service_items	id (PK) service_record_id	Line items: item description, quantity, unit price, item type ('part' or 'labor').
reminders	id (PK) vehicle_id, customer_id	Automated alerts queue: reminder type, message, status ('Pending', 'Sent').

5. Connecting Flask to MySQL: Step-by-Step Code Analysis

The project connects Flask to MySQL through a modular 4-tier pattern. Here is how each module functions:

Step A: Centralized Configuration (config.py)

All database parameters and credentials are abstracted into a single Config class. This allows seamless environment switching (e.g. Docker, production vs local XAMPP):

```
# config.py
import os
class Config:
    SECRET_KEY = os.environ.get('SECRET_KEY', 'fleetsync-secure-key-2026')
    MYSQL_HOST = os.environ.get('MYSQL_HOST', 'localhost')
    MYSQL_PORT = int(os.environ.get('MYSQL_PORT', 3306))
    MYSQL_USER = os.environ.get('MYSQL_USER', 'root')
    MYSQL_PASSWORD = os.environ.get('MYSQL_PASSWORD', '')  # XAMPP default is empty
    MYSQL_DB = os.environ.get('MYSQL_DB', 'fleetsync_db')
```

Step B: The Database Manager (db.py)

Instead of opening raw connections inside individual views, db.py exposes a unified helper query_db(). This design provides critical engineering benefits:

1. **SQL Injection Prevention:** Uses parameterized queries (%s placeholders).
2. **Dictionary Cursors:** Columns are returned as Python dictionaries (e.g., row['name']) rather than tuples.
3. **Transaction Safety:** Automatically executes conn.commit() for inserts/updates, and conn.rollback() if an exception occurs.
4. **Leak Prevention:** Closes cursors and connections inside a finally: block.

```
# db.py (Core execution engine)
def query_db(query, params=(), one=False, commit=False):
    conn = None
    cursor = None
    try:
        conn = get_connection(include_db=True)
        cursor = conn.cursor(dictionary=True)   # Dictionary rows
        cursor.execute(query, params)          # Safe parameterized binding
        if commit:
            conn.commit()
            return cursor.lastrowid or cursor.rowcount
        result = cursor.fetchall()
        return (result[0] if result else None) if one else result
    except Error as e:
        if conn and commit:
            conn.rollback()                  # Rollback on failure
        raise e
    finally:
        if cursor: cursor.close()
        if conn: conn.close()                # Connection safely released
```

Step C: Consuming Data in Flask Routes (app.py)

Flask routes call query_db() to fetch or update data, then pass results into Jinja2 templates. Here are two real examples from the codebase:

```
# Example 1: Secure Login Verification (app.py)
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        email = request.form.get('email', '').strip()
        password = request.form.get('password', '')
        # Parameterized query protects against SQL injection
        user = query_db('SELECT * FROM users WHERE email = %s', (email,), one=True)
        if user and check_password_hash(user['password_hash'], password):
            session['user_id'] = user['id']
            return redirect(url_for('dashboard'))
        flash('Invalid email or password', 'danger')
    return render_template('auth/login.html')

# Example 2: Creating a Vehicle Record with Auto-Commit (app.py)
@app.route('/vehicles/new', methods=['POST'])
@login_required
def vehicles_create():
    new_id = query_db('''
        INSERT INTO vehicles (customer_id, vin, license_plate, make, model, year, odometer)
        VALUES (%s, %s, %s, %s, %s, %s, %s)
    ''', (customer_id, vin, plate, make, model, year, odometer), commit=True)
    flash('Vehicle successfully registered!', 'success')
    return redirect(url_for('vehicle_detail', vehicle_id=new_id))
```

Step D: Global State Injection (Context Processors)

Flask's @app.context_processor runs before any template renders. It queries the database to inject common variables (such as the logged-in user profile and overdue reminder badges) into every template automatically, without needing to pass them manually in every render_template() call.

6. Step-by-Step Manual Execution Guide (Windows)

Whenever you start working on this project, follow this 5-step operational workflow:

Step	Action	Exact Command / Procedure
1	**Start MySQL**	Open **XAMPP Control Panel** → Click **Start** next to MySQL. Ensure port 3306 is highlighted green.
2	**Open Terminal**	Open terminal in your editor (Ctrl + `) and ensure you are in: d:\internship 2
3	**Init Database** *(Once only)*	Run: python setup_db.py Creates database, executes schema.sql, and seeds demo records.
4	**Launch Flask**	Run: python app.py Keep this terminal active! Do not close it while using the website.
5	**Access in Browser**	Navigate to: **http://127.0.0.1:5000** Login with admin@fleetsync.io / Admin@123

7. Troubleshooting & Common Gotchas

Symptom	Root Cause	Resolution
Site Can't Be Reached	Flask web server is not running.	Run python app.py in terminal and leave it running.
Can't connect to MySQL server	XAMPP MySQL service is stopped or port 3306 blocked.	Open XAMPP Control Panel & click Start next to MySQL.
Database 'fleetsync_db' does not exist	Schema has not been seeded yet.	Run python setup_db.py once in terminal.
Address already in use (Port 5000)	A previous Flask instance is still running in background.	Kill port 5000 via PowerShell: Stop-Process -Id (Get-NetTCPConnection -LocalPort 5000).OwningProcess